

Lesson 4 – Insecure Cloud Configuration

An insecurely writable S3 receipts bucket allowed attacker-controlled files to be uploaded and processed by backend receipt logic, creating a dangerous trust boundary violation.

1. Goal and Vulnerability Summary

The goal of this lesson was to demonstrate that the DVSA receipts S3 bucket was insecurely configured and allowed attacker-controlled objects to be uploaded into backend receipt-processing storage. In the intended application workflow, receipt files are generated after payment, stored in the receipts bucket, later processed for shipment, and then returned to the user in final form. However, because the bucket permissions were too permissive, an attacker could write files into this trusted storage location. The affected components in this lesson were the receipts S3 bucket, the downstream Lambda receipt-processing path, and the backend logic that treated bucket content as trusted input. The security impact was that untrusted uploaded files could enter a privileged workflow and trigger backend processing behavior that should have accepted only trusted receipt objects.

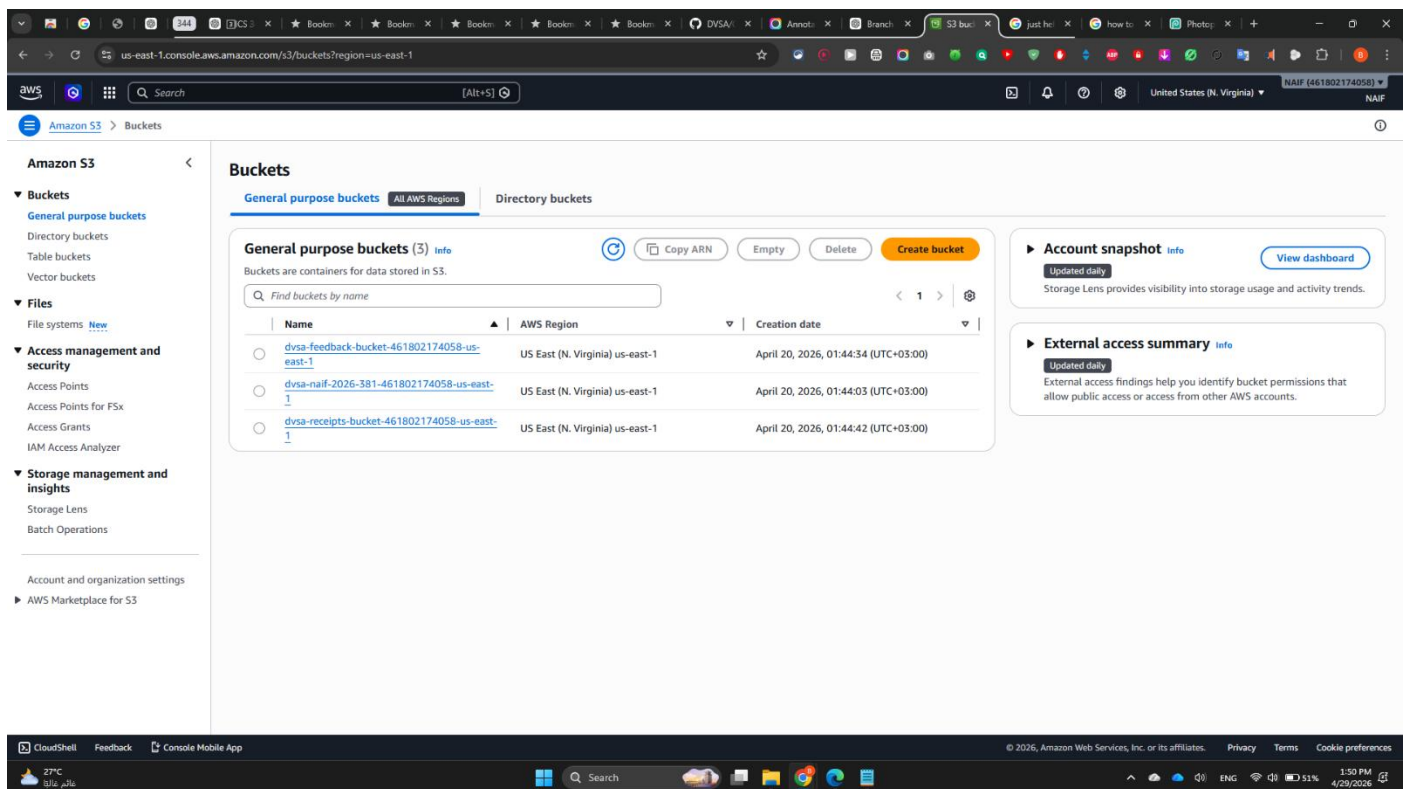


Figure 1. S3 bucket list showing the DVSA receipts bucket.

2. Why This Works / Root Cause

This vulnerability existed because the receipts bucket trusted uploaded content too broadly. The bucket configuration exposed a dangerous write path through its access settings, allowing attacker-controlled files to be placed into a storage location that the application later used as part of the normal receipt-processing workflow. In the official GitHub lesson, this misconfiguration is shown as the foundation for a deeper exploit chain in which attacker-controlled filenames can influence backend processing. In the current DVSA deployment used for this project, the core weakness was still present: the bucket accepted untrusted uploads and the backend receipt flow processed

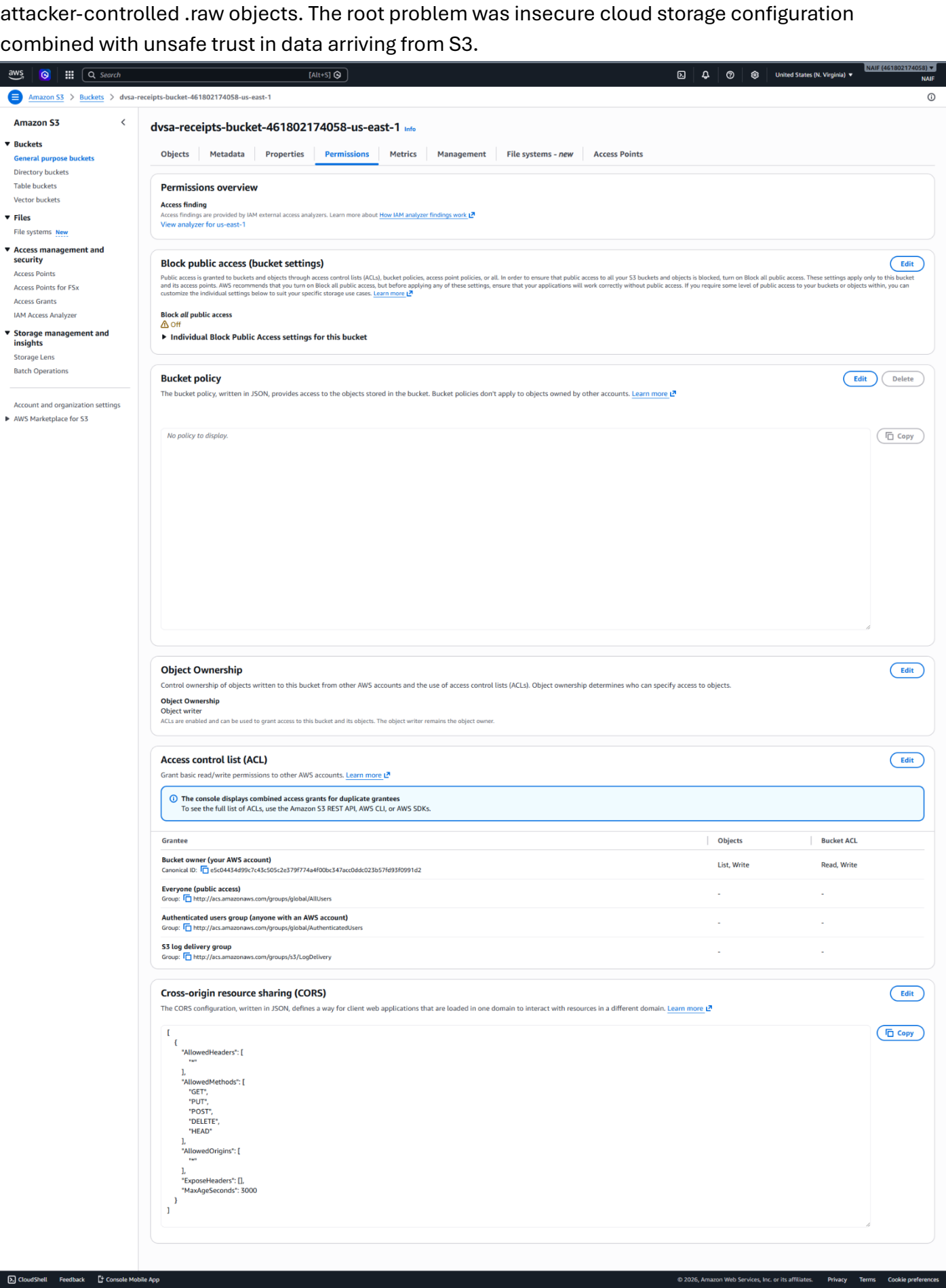


Figure 2. Receipts bucket permissions before remediation showing insecure access settings.

3. Environment and Setup

DVSA was deployed in us-east-1 in a controlled non-production AWS account. Two test users were created and sample orders were placed so that the application produced receipt-related data and date-based receipt paths in the receipts bucket. The main AWS components involved in this lesson were the S3 receipts bucket `dvsa-receipts-bucket-461802174058-us-east-1`, the downstream receipt-processing Lambda function `DVSA-SEND-RECEIPT-EMAIL`, CloudWatch Logs, and CloudShell for controlled upload testing. The official GitHub lesson for Lesson 4 was also reviewed to align the demonstration with the intended vulnerable workflow.

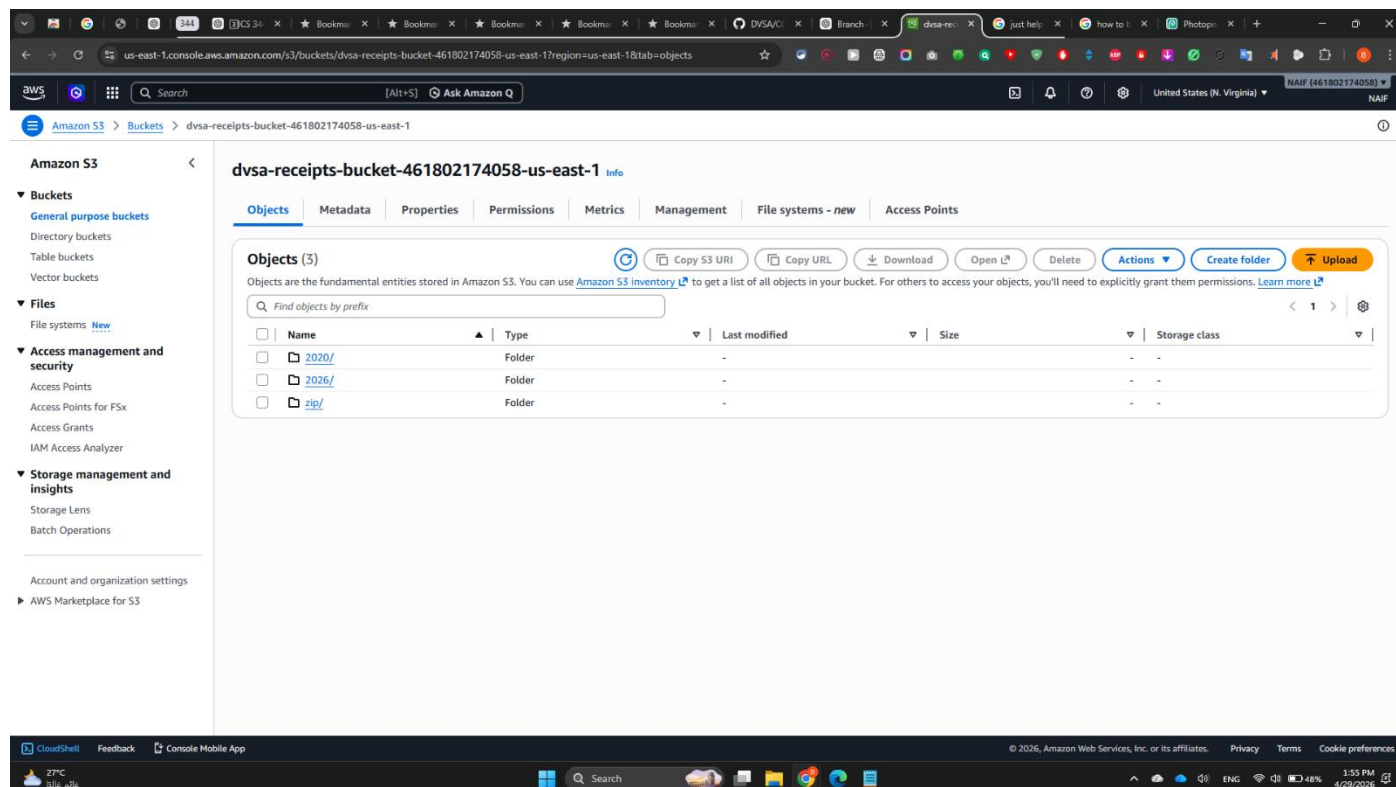


Figure 3. Receipts bucket object structure showing date-based storage paths.

4. Reproduction Steps

- 4.1. The receipts bucket was identified in S3 and its permissions were reviewed.
- 4.2. The bucket configuration showed that Block Public Access was disabled before remediation and that the ACL exposed a broad access model inconsistent with secure receipt storage.
- 4.3. The bucket objects were inspected and date-based receipt paths were observed, matching the official lesson's receipt-storage workflow.
- 4.4. A controlled upload test from CloudShell successfully placed **test1.txt** into the receipts bucket, confirming that attacker-controlled writes to the bucket were possible.

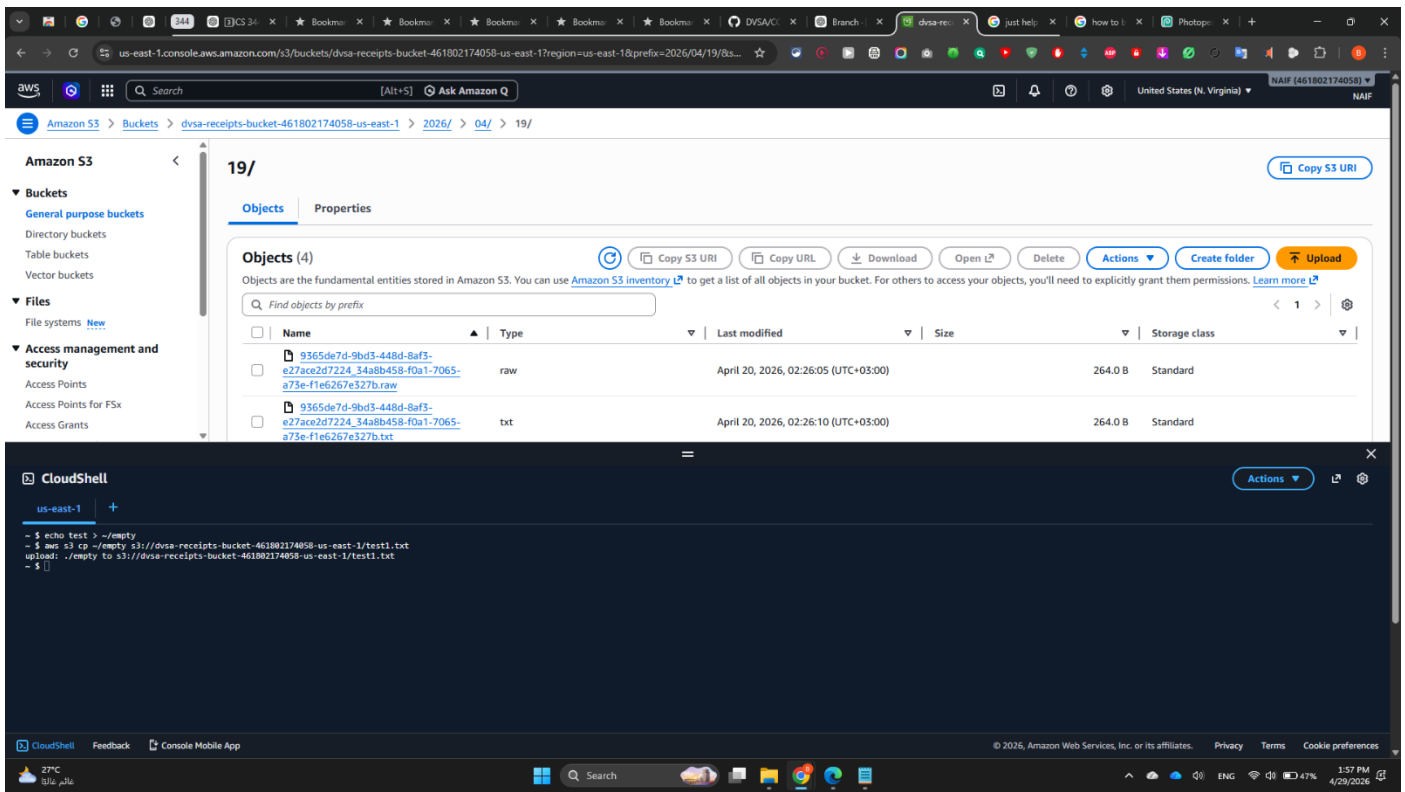


Figure 4. Successful CloudShell upload to the receipts bucket before remediation.

4.5. A second controlled upload placed a .raw object into the date-based receipt path.

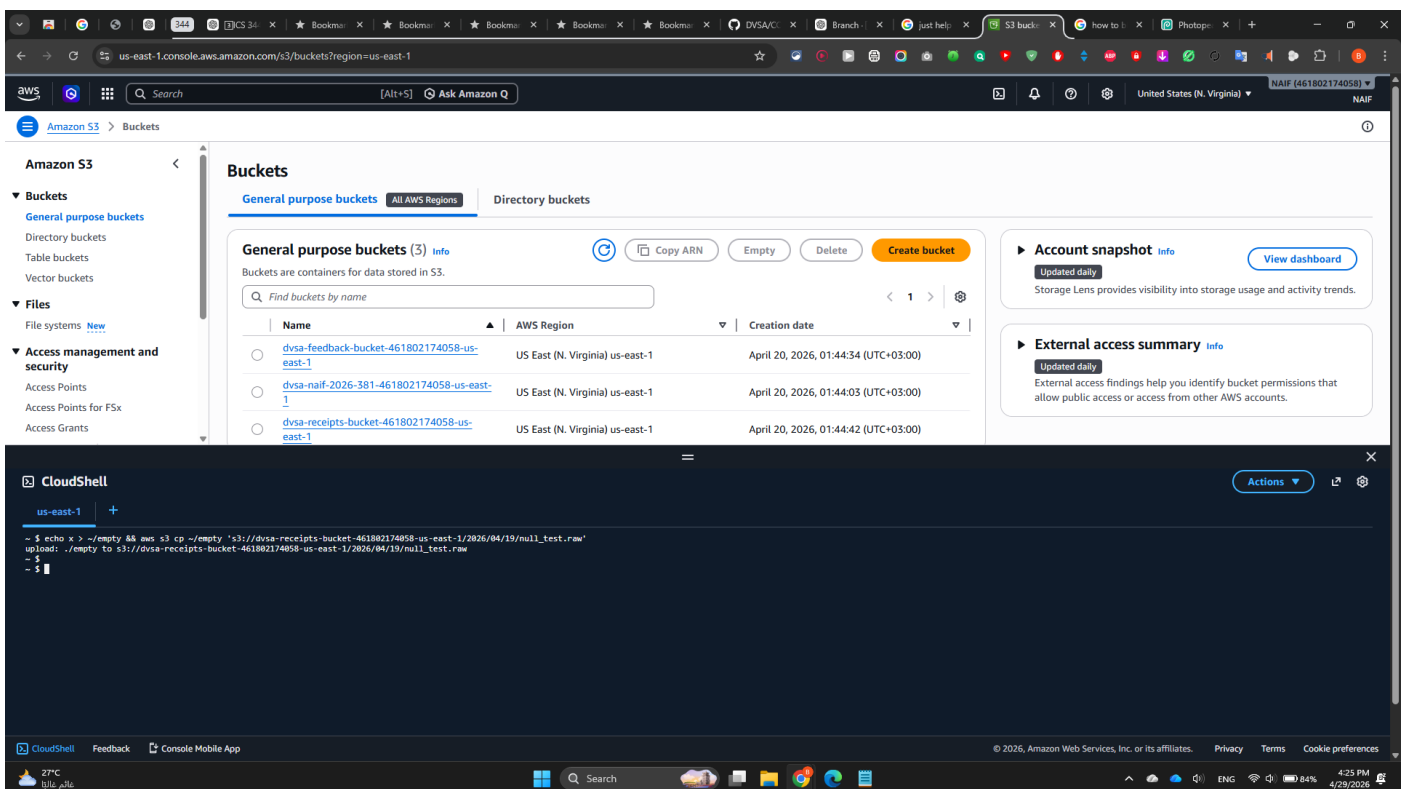


Figure 5. Controlled .raw upload to the date-based receipt path.

4.6. After refresh, a corresponding .txt object appeared in the same path, showing that backend receipt-processing logic had acted on the uploaded attacker-controlled file.

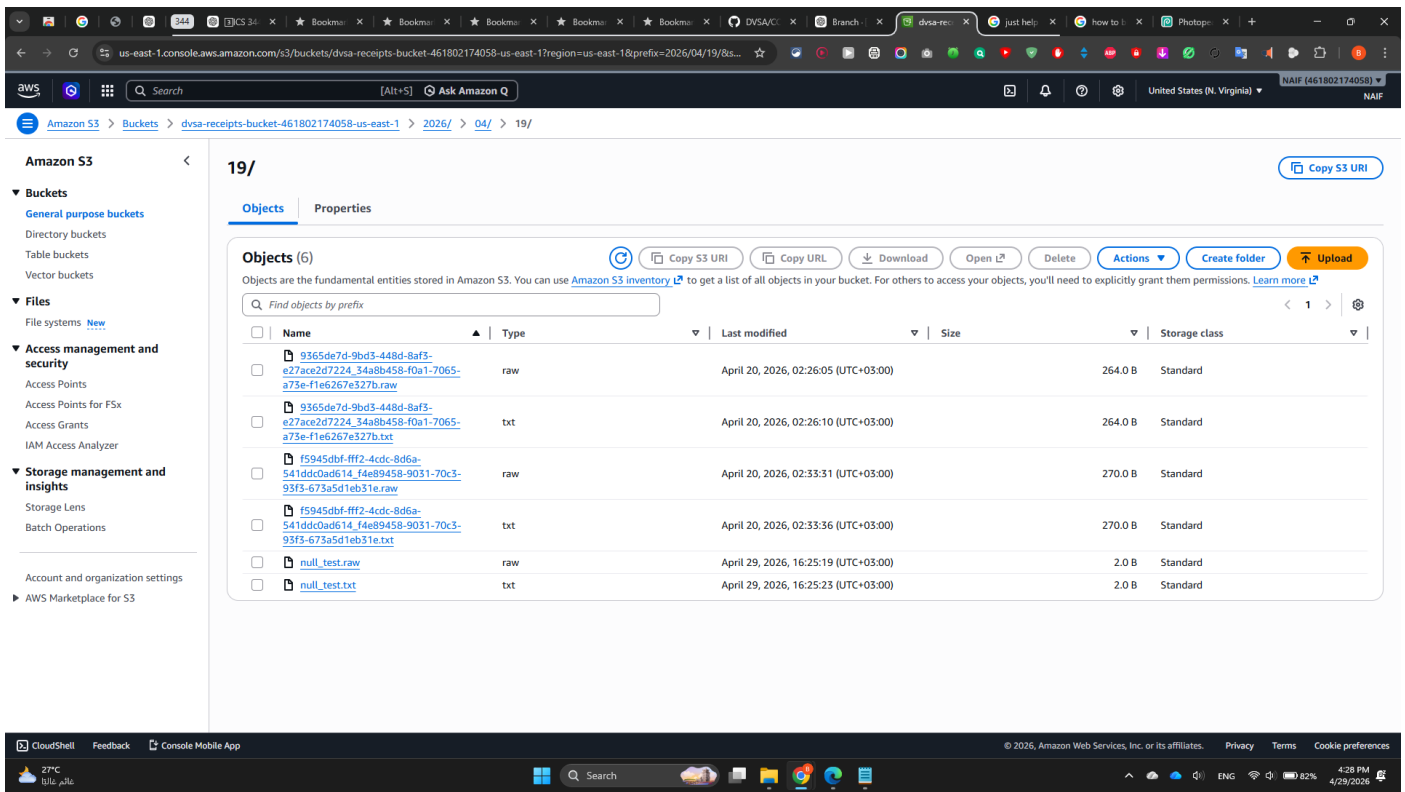


Figure 6. Date-based receipt path after refresh showing attacker-controlled .raw input and generated .txt output.

4.7. CloudWatch logs for DVSA-SEND-RECEIPT-EMAIL were reviewed and showed that the Lambda function was triggered at the time of the upload.

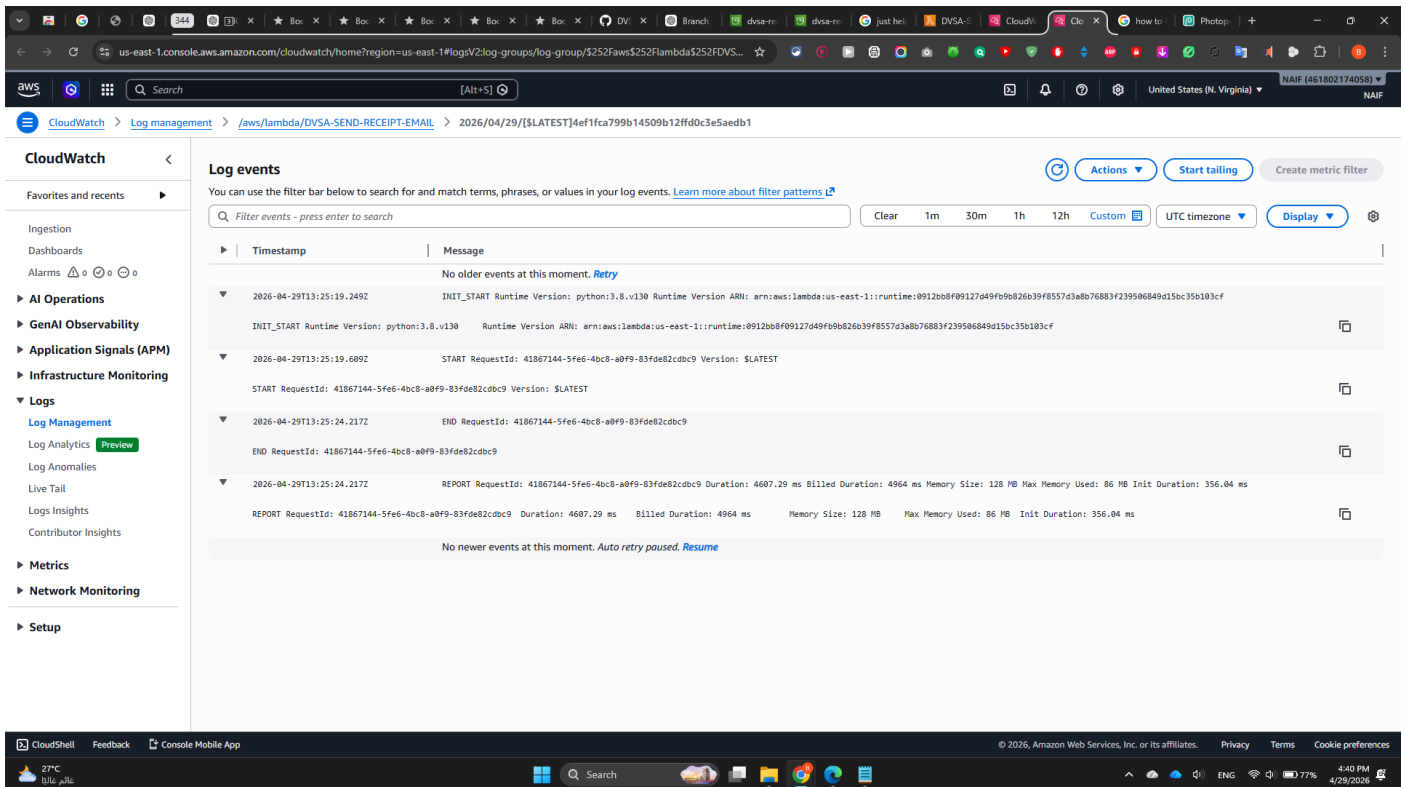


Figure 7. CloudWatch log stream for DVSA-SEND-RECEIPT-EMAIL showing Lambda execution after the upload.

4.8. These steps confirmed the insecure cloud configuration described in the official lesson: untrusted files could be uploaded into the receipts bucket and then enter trusted backend processing.

5. Evidence and Proof

The vulnerability was proven through multiple sources of evidence. First, the S3 permissions view showed that the receipts bucket had insecure settings before remediation, including disabled Block Public Access and an ACL configuration inconsistent with strict receipt storage controls. Second, a direct CloudShell upload to the receipts bucket succeeded, showing that attacker-controlled write access was possible. Third, a **.raw** object uploaded to the date-based receipt path resulted in a new **.txt** object appearing in the same location, proving that downstream application processing acted on attacker-supplied bucket content. Fourth, CloudWatch logs for **DVSA-SEND-RECEIPT-EMAIL** showed Lambda execution at the matching time, confirming that the backend function was triggered by the uploaded object. Together, these findings demonstrated that untrusted S3 content could enter and influence the receipt-processing workflow.

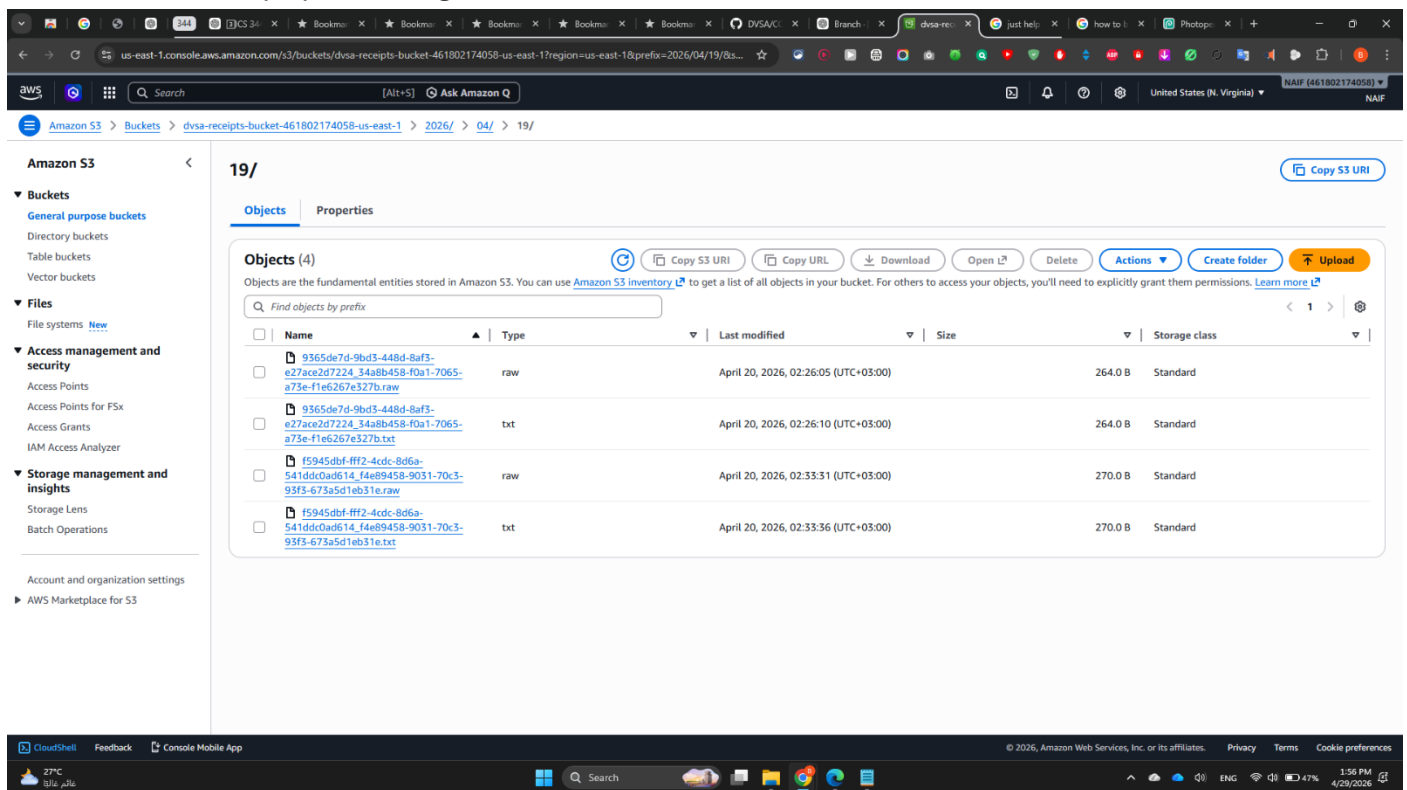


Figure 8. Raw and processed receipt files in the same date path, confirming downstream handling of uploaded content.

6. Fix Strategy / Probable Mitigation

The remediation goal was to harden the receipts bucket so that attacker-controlled objects could no longer be written through the insecure access model shown before the fix. The main mitigation steps were to enable Block Public Access and remove the overly broad ACL exposure from the bucket configuration. Receipt storage should accept only trusted application writes and should not expose a general write path through insecure bucket permissions. In addition, backend receipt-processing logic should treat bucket-derived content defensively and should not trust filenames or object content arriving from S3 without validation.

7. Code / Config Changes

Two configuration changes were applied during remediation. First, Block Public Access was enabled for the receipts bucket. Second, the bucket ACL configuration was cleaned so that the insecure broad

access model observed before remediation was removed from the bucket's effective permissions state. These changes reduced the risk that attacker-controlled files could be written into trusted receipt-processing storage.

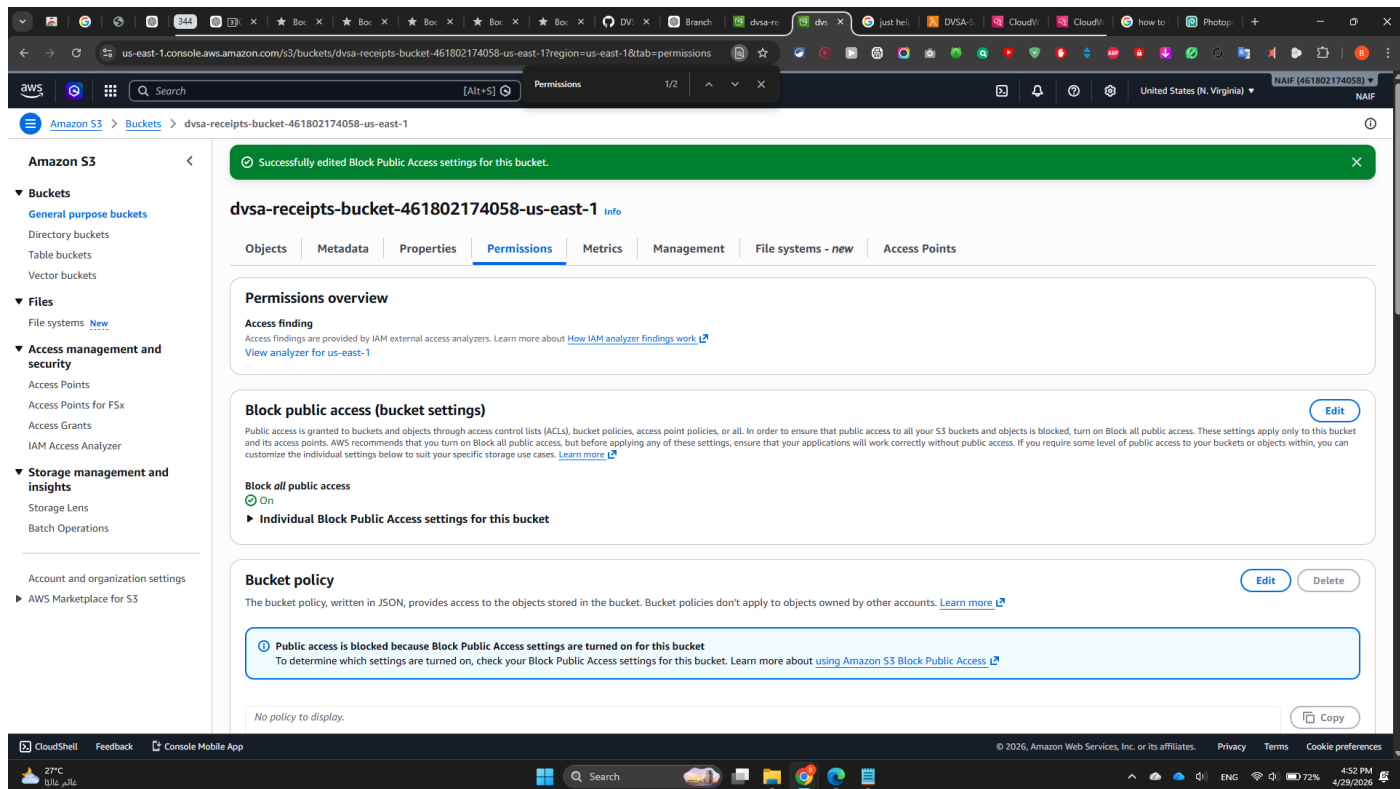


Figure 9. Block Public Access enabled for the receipts bucket during remediation.

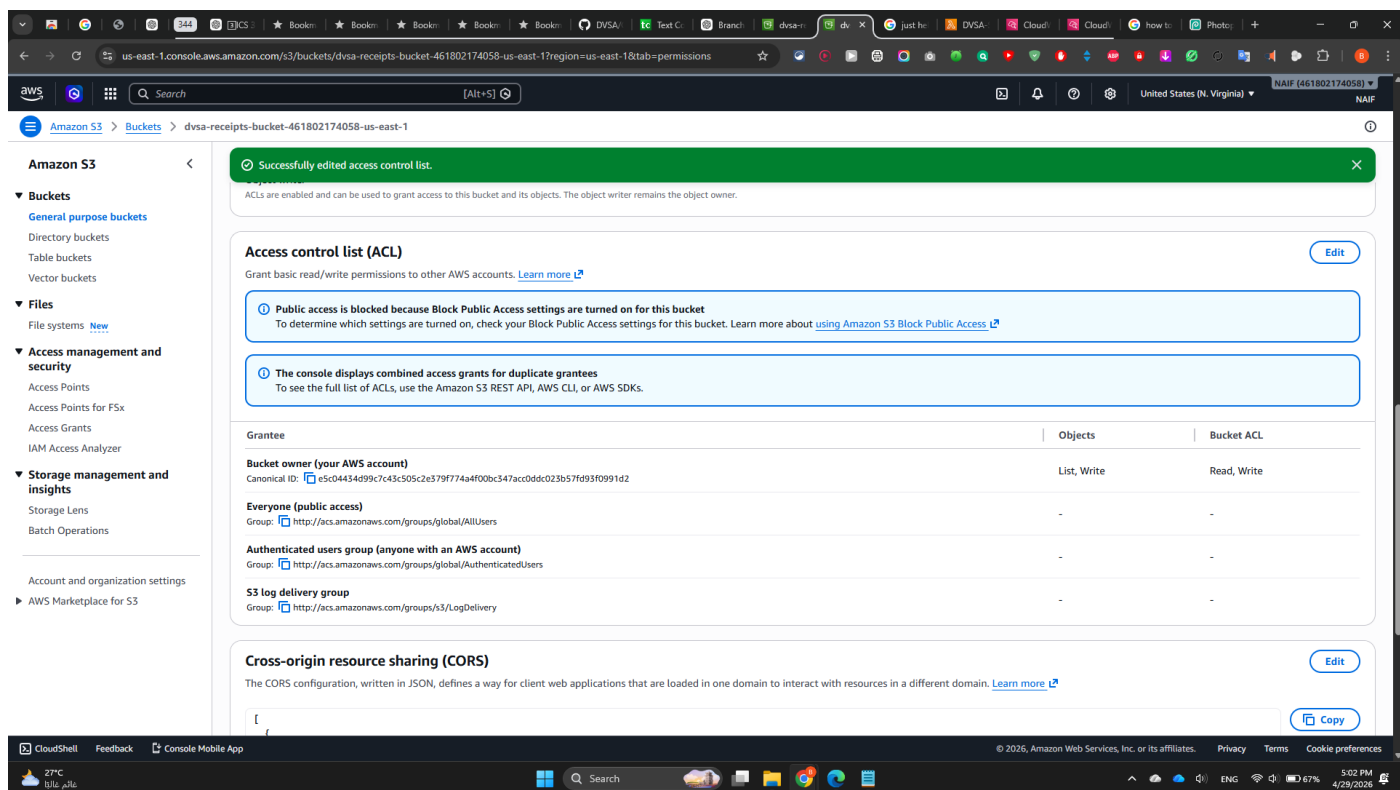


Figure 10. Bucket ACL remediation removing the insecure broad access exposure.

8. Verification After Fix

After remediation, the receipts bucket permissions were reviewed again. Block Public Access was enabled, and the bucket's post-fix permissions view showed a hardened configuration compared with the earlier vulnerable state. Because the validation in this project was performed from the same AWS account that owned the bucket, same-account write behavior was not treated as proof of public or cross-account exposure after the fix. Therefore, the post-fix verification focused on the corrected bucket permissions state: public access blocking was enabled and the previously insecure broad ACL exposure observed before remediation had been removed from the effective permissions state used in the vulnerable setup. These checks confirmed that the insecure cloud configuration had been

reduced and that the bucket was in a stronger security state after remediation.

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

AWS Marketplace for S3

dvsa-receipts-bucket-461802174058-us-east-1

Info

ObjectsMetadataPropertiesPermissionsMetricsManagementFile systems - newAccess Points

Permissions overview

Access finding

Access findings are provided by IAM external access analyzers. Learn more about [How IAM analyzer findings work](#).

[View analyzer for us-east-1](#)

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

On

Individual Block Public Access settings for this bucket

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

Public access is blocked because Block Public Access settings are turned on for this bucket

To determine which settings are turned on, check your Block Public Access settings for this bucket. Learn more about [using Amazon S3 Block Public Access](#)

No policy to display.

Copy

Object Ownership

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

Object writer

ACLs are enabled and can be used to grant access to this bucket and its objects. The object writer remains the object owner.

Access control list (ACL)

Grant basic read/write permissions to other AWS accounts. [Learn more](#)

Public access is blocked because Block Public Access settings are turned on for this bucket

To determine which settings are turned on, check your Block Public Access settings for this bucket. Learn more about [using Amazon S3 Block Public Access](#)

The console displays combined access grants for duplicate grantees

To see the full list of ACLs, use the Amazon S3 REST API, AWS CLI, or AWS SDKs.

Grantee	Objects	Bucket ACL
Bucket owner (your AWS account) Canonical ID: e5c04454899c7c43c505c2e379f774a4f00bc347acc0ddc023b57fd93f0991d2	List, Write	Read, Write
Everyone (public access) Group: http://acs.amazonaws.com/groups/global/AllUsers	-	-
Authenticated users group (anyone with an AWS account) Group: http://acs.amazonaws.com/groups/global/AuthenticatedUsers	-	-
S3 log delivery group Group: http://acs.amazonaws.com/groups/s3/LogDelivery	-	-

Cross-origin resource sharing (CORS)

The CORS configuration, written in JSON, defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. [Learn more](#)

```
{  "AllowedHeaders": [    "x-amz-acl"  ],  "AllowedMethods": [    "GET",    "PUT",    "POST",    "DELETE",    "HEAD"  ],  "AllowedOrigins": [    "*"  ],  "ExposeHeaders": [],  "MaxAgeSeconds": 3000}
```

Copy

CloudShellFeedbackConsole Mobile App

© 2016, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

Figure 11. Post-fix permissions state showing the hardened receipts bucket configuration.

9. Structured Operation and Security Analysis

A. Intended Logic and Security Rule(s):

The receipts bucket should store only trusted receipt objects created by the DVSA application workflow. Backend receipt-processing functions should operate only on valid application-generated receipt files and must not treat attacker-controlled S3 objects as trusted input.

B. Evidence Sources and Behavior Trace:

The analysis relied on S3 bucket permissions, ACL state, bucket object paths, CloudShell upload results, generated **.txt** side effects, and CloudWatch logs from **DVSA-SEND-RECEIPT-EMAIL**.

C. Normal vs. Exploit Behavior:

Under normal behavior, only trusted application logic should place receipt objects into the bucket, and only valid receipt files should be processed by the backend. Under exploit conditions, attacker-controlled objects were uploaded into the receipts bucket and then entered downstream processing, as shown by the generated **.txt** file and matching Lambda execution.

D. Why This Is a Security Deviation / Fix / Verification:

This is a security deviation because trusted backend receipt processing should not accept untrusted files from an insecurely writable bucket. The fix hardened the bucket configuration by enabling Block Public Access and cleaning the insecure ACL exposure. Verification confirmed a safer post-fix permissions state than the vulnerable configuration shown before remediation.

10. Takeaway / Lessons Learned

This lesson shows that insecure cloud configuration can turn storage into an attack surface. Even without directly compromising application code, an attacker may abuse weak S3 permissions to inject untrusted objects into a trusted backend workflow. In serverless systems, bucket permissions are part of the security boundary and must be reviewed with the same care as code. Receipt-processing flows should trust only validated application-generated objects, and S3 permissions should be restricted to the minimum required access.